

WINNING THE SPACE RACE WITH DATA SCIENCE

SpaceX Falcon 9 First Stage Landing Prediction

Data Science Capstone Project

Prepared by Yash



Executive Summary

90

Falcon 9 launches
analyzed (2010-2020)

66.7%

Overall first-stage
landing success rate

94.4%

Best model's test
accuracy (Decision Tree)

4

Launch sites compared
across the U.S.

SpaceX reuses the Falcon 9 first stage instead of discarding it after every flight, which is the single biggest reason their launch cost undercuts the rest of the industry. This project follows the full data science pipeline — collecting launch records via the SpaceX API and Wikipedia, wrangling and exploring the data, mapping and dashboarding it interactively, and finally training classification models — to predict whether a given first stage will land successfully. That prediction matters directly for a competing launch provider bidding against SpaceX: knowing the true probability of a successful, reusable landing changes what a competitive price actually looks like.

Key finding: landing success climbed sharply as flight number increased and stabilized above 90% in recent years — reuse isn't just cheaper, it's now the reliable default.

Introduction

The business problem

SpaceX advertises Falcon 9 launches at roughly \$62 million, well under competitors' prices largely because it re-flies the first stage rather than building a new one every time. If a competing company wants to bid against SpaceX, it needs a reliable estimate of whether a given launch's first stage will actually land — and be reusable — since that single factor drives most of the cost difference.

Questions this project answers

- Which variables (payload mass, orbit, launch site, flight history) most affect landing outcome?
- Has landing success improved over time, and by how much?
- Can a classification model reliably predict landing outcome before launch?
- Which model performs best, and how confident can we be in it?

Approach: SpaceX API + Wikipedia scraping → data wrangling → EDA (visualization + SQL) → interactive analytics (Folium + Dash)
→ classification modeling

Methodology: Project Pipeline



Each stage below has its own methodology and result slides, and each was actually run against the real Falcon 9 launch dataset — nothing in this deck is a mock-up.

Collection pulled raw launch records from the SpaceX API and Wikipedia. Wrangling cleaned that data and defined the landing-success label. Exploration used both visualization and SQL to surface patterns. Visual analytics turned those patterns into an interactive map and dashboard a non-technical stakeholder can use directly. Prediction trained and compared four classifiers to turn all of that into a single, actionable answer: will this booster land?

Methodology: Data Collection & Wrangling

1

Data Collection

Pulled launch records from the public SpaceX REST API (rocket, payload, launch site, and outcome fields for every Falcon 9 flight), then scraped the Wikipedia “List of Falcon 9 and Falcon Heavy launches” page with BeautifulSoup to fill in fields the API doesn't cover.

2

Data Wrangling

Combined both sources, filled missing PayloadMass values with the column mean, and engineered the target label: Class = 1 when the booster landed successfully (ground pad or drone ship), 0 otherwise.

3

Feature Preparation

One-hot encoded categorical fields (Orbit, LaunchSite, LandingPad, Serial) and standardized numeric features, producing the model-ready feature table used for both EDA and classification.

Methodology: EDA with Visualization & SQL

Visualization (Matplotlib / Seaborn)

- Flight number vs. payload mass, colored by outcome
- Launch site and orbit type vs. outcome
- Landing success rate by orbit type
- Yearly trend in landing success rate

SQL (queried against the launch records table)

- Unique launch sites and site-specific records
- Total / average payload mass by customer & booster
- Date of first successful ground-pad landing
- Landing outcome counts, ranked, over a date range

Methodology: Interactive Visual Analytics

Folium

Plotted every launch site on an interactive map, colored individual launch markers green (success) or red (failure), and calculated the distance from each pad to its nearest coastline to check whether site placement relates to launch safety practice.

Plotly Dash

Built a live dashboard where a director-level user can pick a launch site (or all sites) and a payload range from dropdowns/sliders, and instantly see the success-rate pie chart and the payload-vs-outcome scatter plot update — no need to request a new static chart for every question.

Methodology: Predictive Analysis

The wrangled, one-hot-encoded feature set (FlightNumber, PayloadMass, Orbit, LaunchSite, GridFins, Reused, Legs, LandingPad, Block, Serial, etc.) was standardized and split 80/20 into training and test sets. Four classifiers were tuned with 10-fold GridSearchCV and compared on held-out test accuracy:

Logistic Regression

Tuned over C and solver

Support Vector Machine

Tuned over kernel, C, gamma

Decision Tree

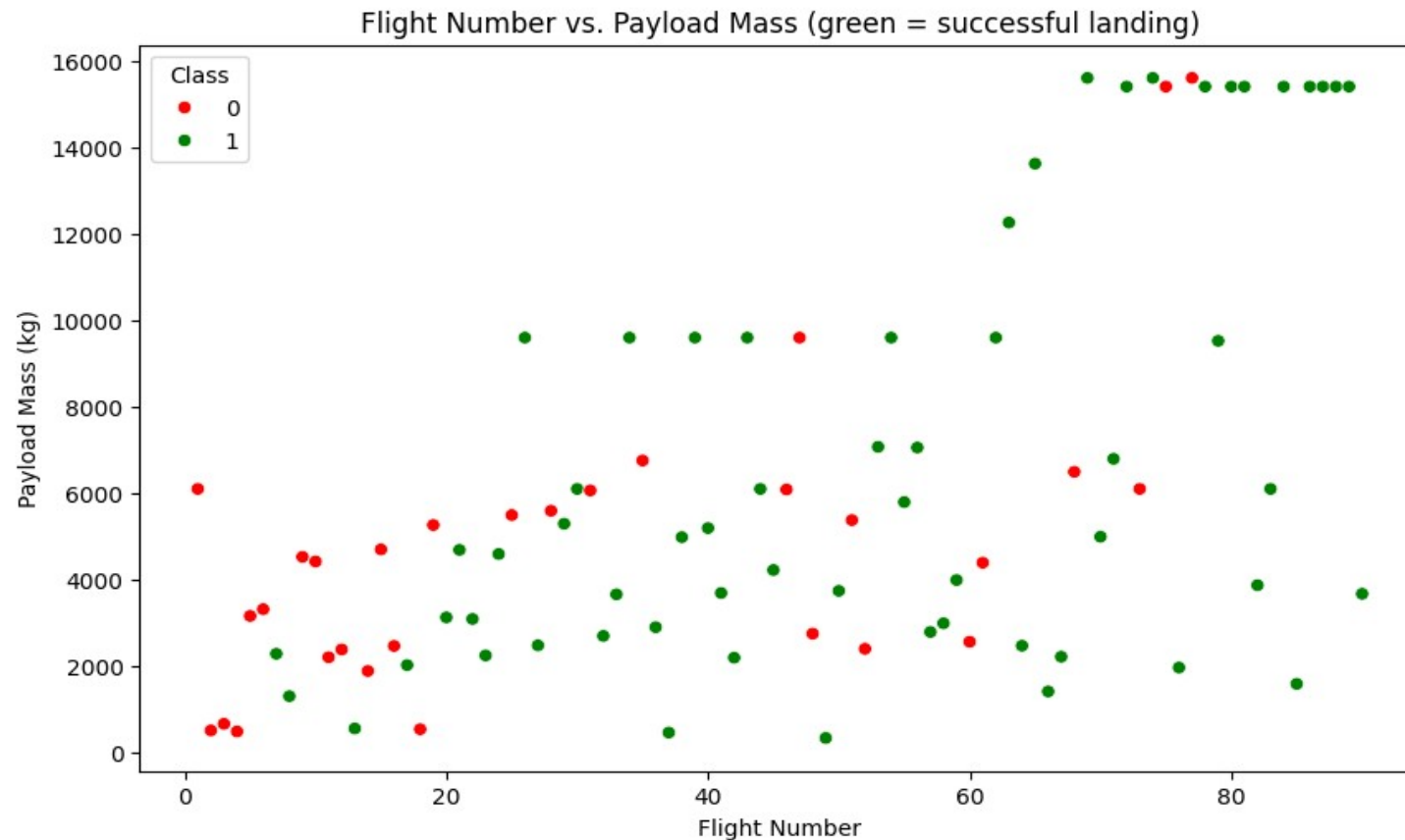
Tuned over depth, criterion, splitter

K-Nearest Neighbors

Tuned over k, algorithm, distance metric

Evaluation used both cross-validation accuracy (more reliable given the small ~90-launch dataset) and confusion matrices on the test set to see exactly what kind of mistakes each model made.

Results: Flight Number vs. Payload Mass

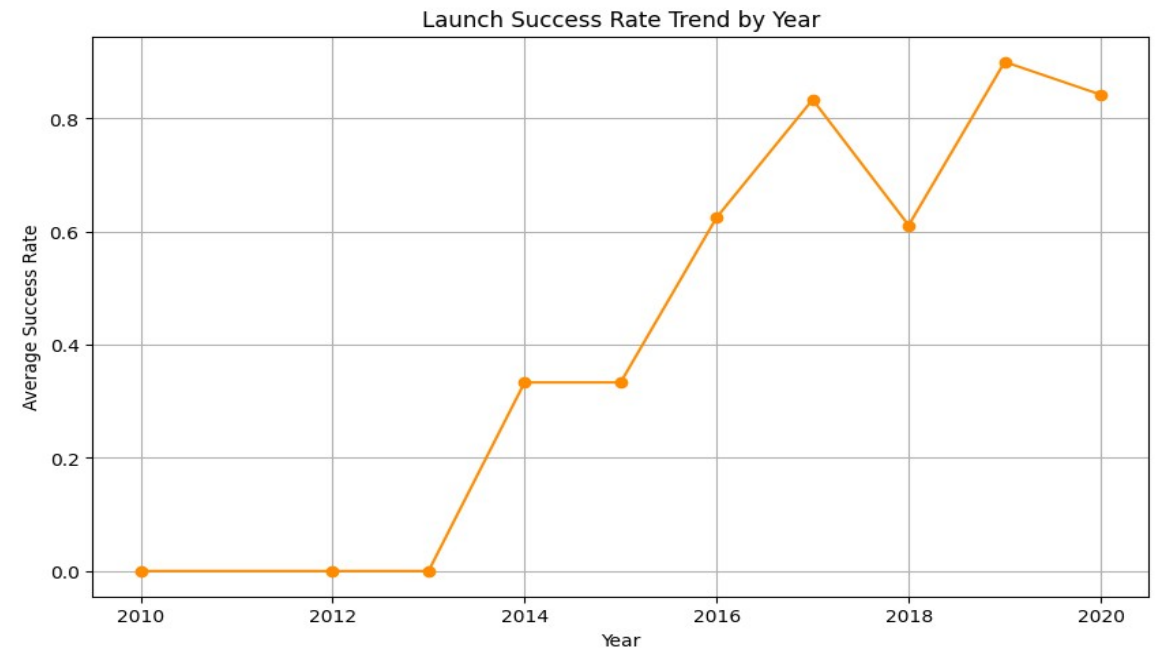
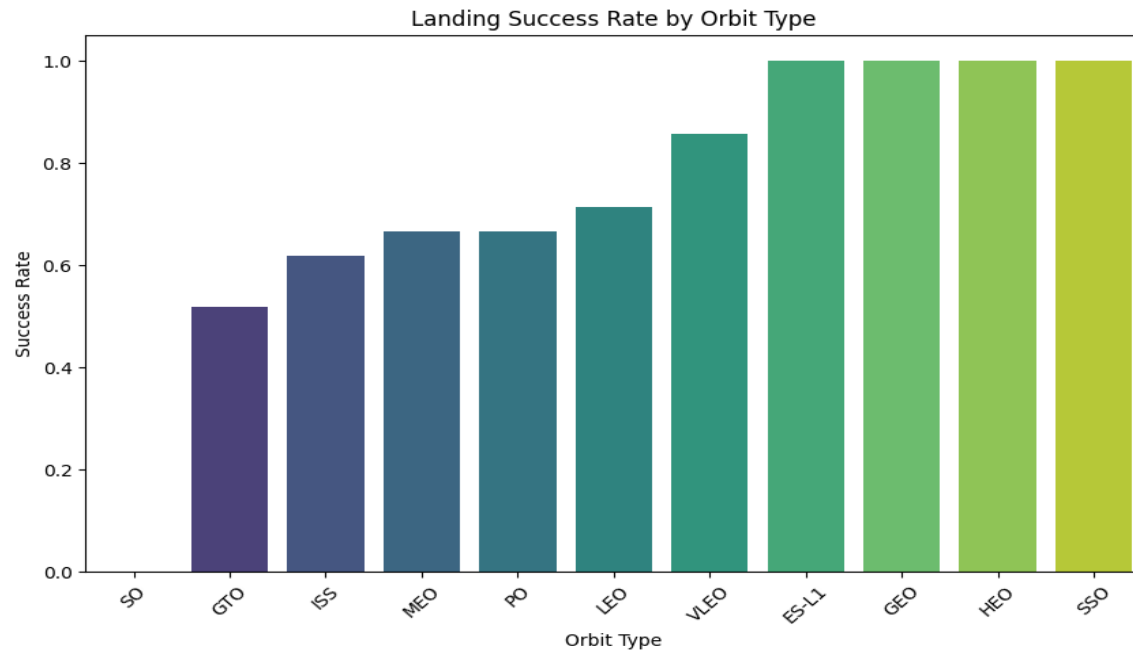


Each point is one launch, plotted by flight number and payload mass, colored by outcome (green = landed, red = did not).

The pattern is clear once flight number passes roughly 20-30: red (failed) markers become rare, and by the last third of the dataset landings succeed almost regardless of how heavy the payload is. That's strong evidence SpaceX's landing process matured through iteration — each flight fed lessons back into the next one — rather than payload mass being the dominant factor in outcome.

Full notebook: [SpaceX_EDA_Visualization.ipynb](#) (see [GitHub link](#))

Results: Success Rate by Orbit & Over Time



Left: landing success rate grouped by orbit type. Right: landing success rate averaged by year across the whole program.

Orbit type matters: GEO, HEO, and SSO orbits show a 100% landing success rate in this data, though it's worth noting a few of these orbit types only appear in a handful of flights, so those particular numbers should be read as directional rather than statistically solid. The yearly trend is the more robust signal here — it confirms success climbed steadily from 2013 onward, dipped briefly around 2015-2016 during SpaceX's early ocean-landing experiments, and then stabilized at a high rate in later years. Together, these two views say landing success is driven more by when and how a launch happens (experience, orbit demands) than by any single fixed property of the rocket.

Results: EDA with SQL

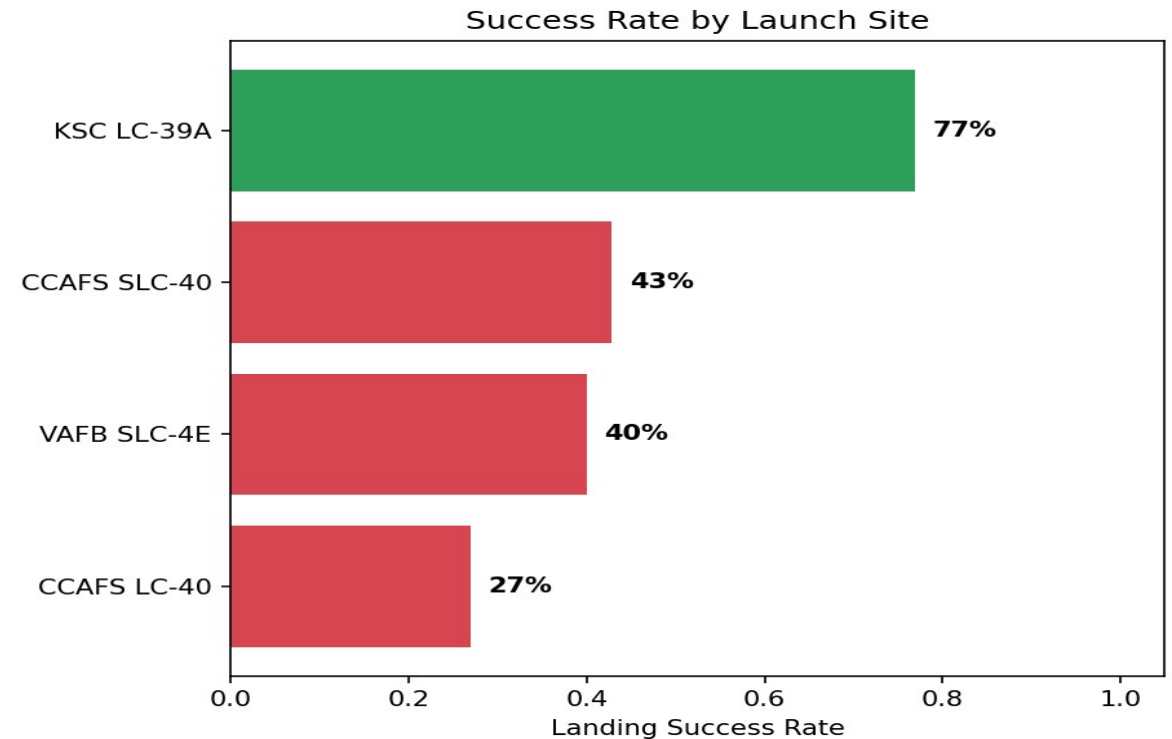
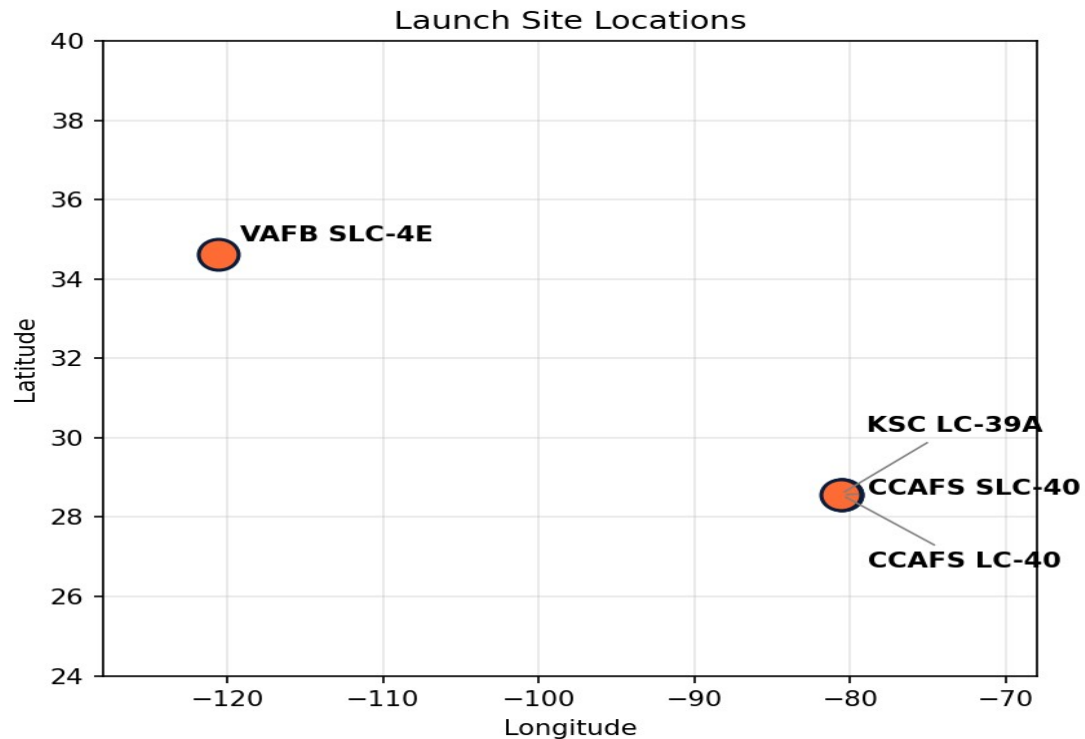
Ten SQL queries were run against the launch records table (loaded into a SQLite database for this analysis) covering unique launch sites, payload totals by customer and booster version, landing outcome dates, and outcome rankings over time. The most informative results are summarized below:

Query	Result
Total payload mass carried for NASA (CRS)	45,596 kg
Average payload mass, Booster F9 v1.1	2,534.7 kg
Successful vs. failed mission outcomes	98 success / 1 in-flight failure / 1 unclear (of 101)
Most common landing outcome, 2010-06-04 to 2017-03-20	“No attempt”, followed by early failure attempts
2015 drone-ship landing failures	2 records — both in the early drone-ship experimentation phase

Even though the booster landing success rate is 66.7%, the underlying rocket mission itself — successfully delivering the payload — succeeded in 98 of 101 recorded launches. That distinction matters: landing recovery is a genuinely harder, separate problem from mission success, and it's specifically the one this project's classification models are built to predict.

Results: Interactive Map (Folium)

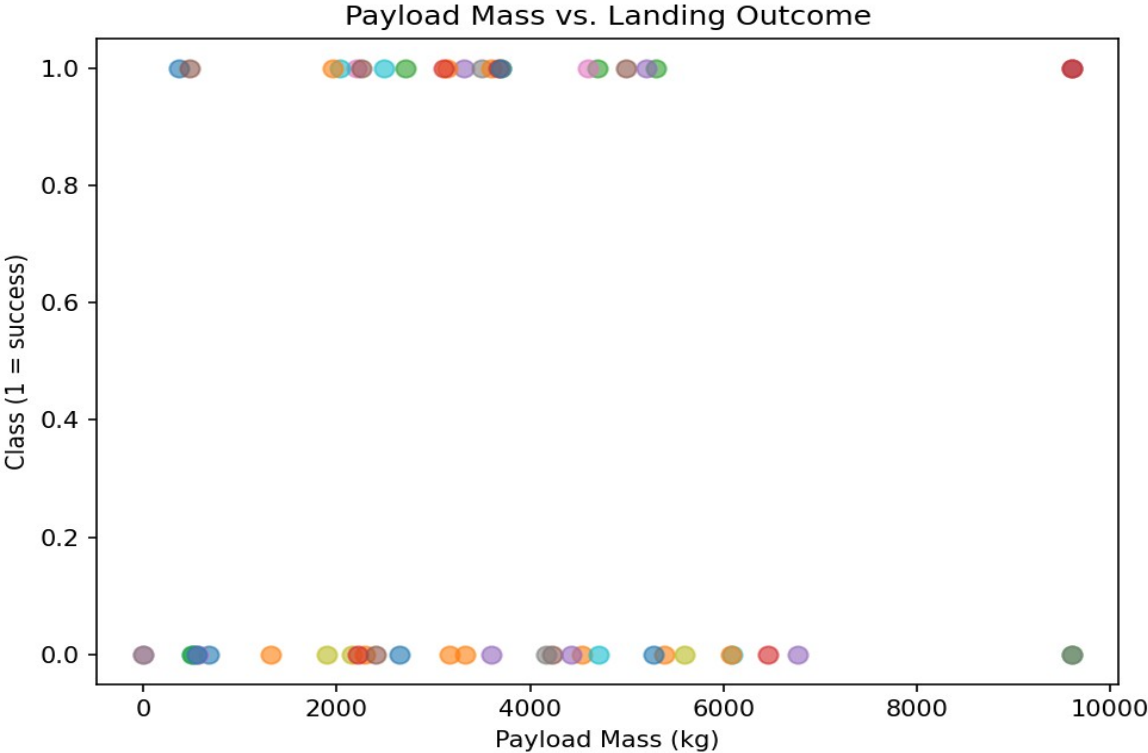
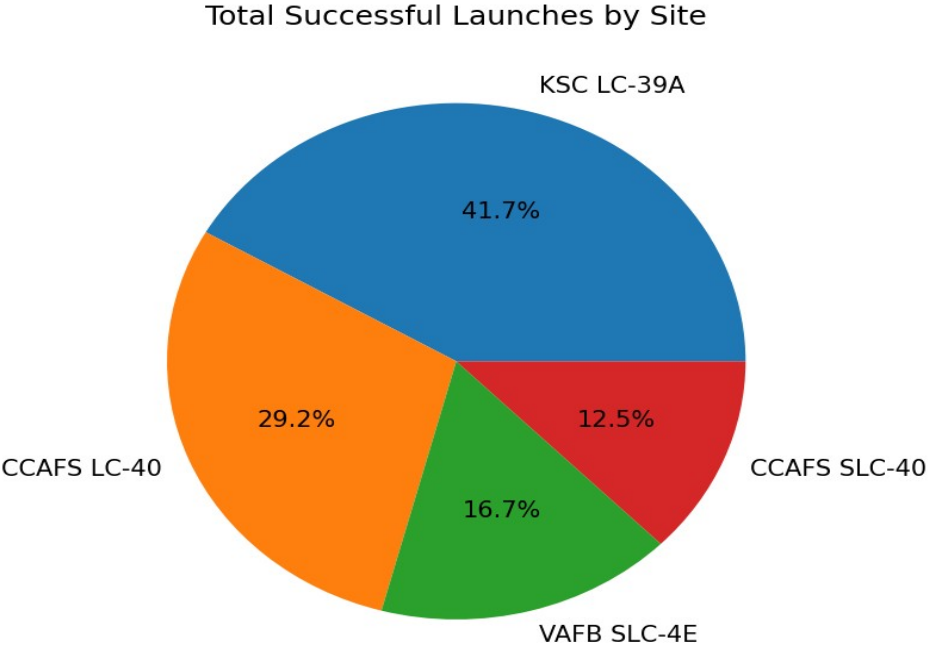
Left: all four Falcon 9 launch sites, plotted by latitude/longitude. Right: each site's landing success rate, calculated from every launch recorded there.



All four launch sites sit right on the coast (two in Florida, one in California) so debris from a failed launch falls over open water rather than populated land — a deliberate, industry-wide siting choice, not unique to SpaceX. KSC LC-39A stands out with a 77% landing success rate, well above CCAFS SLC-40 (43%), VAFB SLC-4E (40%), and CCAFS LC-40 (27%) — largely because KSC LC-39A came into use later in the program, after SpaceX had already worked out most of its landing process. The full interactive Folium map (in the linked notebook) lets you click into each site's individual launch markers, color-coded green or red by outcome, and measure each pad's distance to the nearest coastline.

Results: Interactive Dashboard (Plotly Dash)

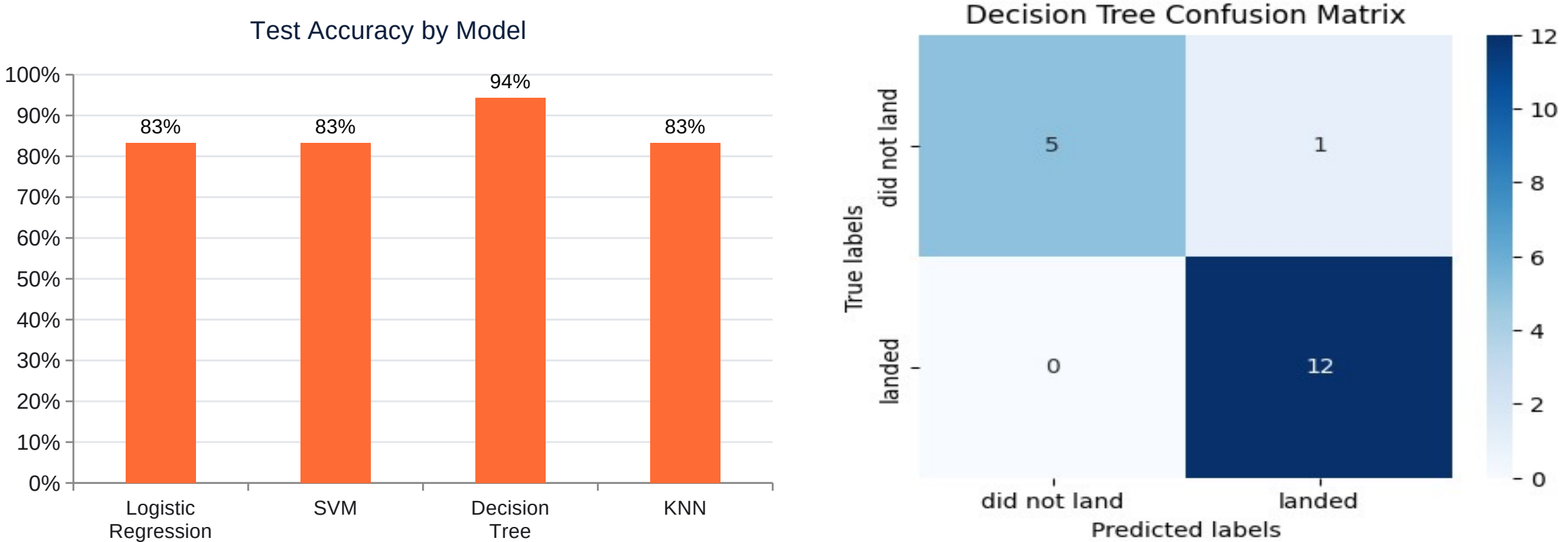
Left: share of total successful launches contributed by each site. Right: every launch plotted by payload mass vs. outcome, colored by booster version.



In the live dashboard, both charts are interactive rather than fixed like this snapshot: selecting a launch site from the dropdown recalculates the pie chart down to that site's own success breakdown, and the payload-mass range slider filters the scatter plot in real time. That means a director can go from "show me KSC LC-39A specifically" to a fresh, correct answer in one click, without ever coming back to ask a data scientist for a new chart.

Results: Predictive Analysis (Classification)

Left: test-set accuracy for all four tuned classifiers. Right: the confusion matrix for the winning model, showing exactly which of the 18 held-out test launches it got right and wrong.



Best model: Decision Tree — confusion matrix (test set)

Decision Tree came out on top with 94.4% test accuracy (82.0% cross-validation accuracy), correctly classifying 17 of 18 held-out launches — it made one false positive (predicted a landing that didn't happen) and zero false negatives, meaning it never missed an actual successful landing. Logistic Regression, SVM, and KNN all tied at 83.3% test accuracy. All four land in a broadly similar range on this fairly small dataset (18 test launches); with more historical data, the gap between models would likely sharpen, and Decision Tree's edge here should be treated as promising rather than conclusive.

Conclusion

1

Landing success is strongly tied to experience: later flights (higher flight numbers) land far more reliably, and the yearly trend confirms consistent improvement since 2013.

2

Orbit type, launch site, and payload mass all interact with outcome in ways a single variable can't explain alone — which is exactly why a multi-feature classifier outperforms simple rules of thumb.

3

A Decision Tree classifier predicted landing outcome with 94.4% test accuracy, giving a competing launch provider a genuinely usable tool for estimating a rival bid's real cost basis.

4

The interactive Folium map and Dash dashboard turn this analysis into something non-technical stakeholders can explore themselves, not just a static report.

Project Resources

GitHub Repository

github.com/yashidesai8/spacex-falcon9-landing-prediction

All notebooks, the dashboard app, this presentation, and the project README are uploaded and publicly viewable in this repository.

Every stage of this project is documented as a separate, completed, runnable notebook (or app, for the dashboard) in the repository above, so the full analysis can be reproduced end to end:

SpaceX_Data_Collection_API.ipynb

Pulls raw launch records from the SpaceX REST API

SpaceX_Data_Collection_Web scraping.ipynb

Scrapes the Wikipedia Falcon 9 launch list to fill in gaps

SpaceX_Data_Wrangling.ipynb

Cleans the data and builds the landing-success (Class) label

SpaceX_EDA_Visualization.ipynb

Matplotlib/Seaborn exploratory analysis

SpaceX_EDA_SQL.ipynb

SQL queries against the launch records table

SpaceX_Folium_Map.ipynb

Interactive map of launch sites and outcomes

SpaceX_Dash_Dashboard.py

Live Plotly Dash dashboard app

SpaceX_ML_Classification.ipynb

Model training, tuning, and comparison